

1. Agent Playtest 0009

Field	Value
Date	2026-09-23 16:00 – 16:40 UTC
Commit	5e71215 “Implement Skein mesh primitives for agent testing” — HEAD throughout
Agent	opcode session, GLM-5.3-Flash, driving the tool API over loopback HTTP
Client	target/debug/client --mcp, BEVY_ASSET_ROOT=\$PWD/playtest_assets/playtest_0009/client
Server	target/release/server, BEVY_ASSET_ROOT=\$PWD/playtest_assets/playtest_0009/server
Level	levels/playtest.level.ron (this playtest’s own level)
Asset isolation	server: 100% plaintext (one .level.ron + one .gltf). client: all content plaintext (five .gltf scenes, the manifest, config, locale); only engine furniture (2 font slots on one TTF, 2 WAVs, 1 KTX2 skybox, 2 atlas PNGs) copied from client/assets
Transports	game: UDP/netcode :6000 · QA tool API: BRP HTTP :15702 (+ MCP :15710)

2. Purpose

The project owner’s idea: make playtests **reproducible** by isolating each playtest’s assets — an agent writes the server assets into playtest_assets/playtest_NNNN/server/ and the client assets into playtest_assets/playtest_0009/client/, all as **plaintext**: 3D content as plain JSON .gltf (not binary .glb), game logic as Skein components in the glTF extras, and mock visuals via the new MeshPrimitive component (5e71215) so **zero baked mesh data** is needed anywhere. Fonts excluded on the theory that Bevy’s built-in default covers ASCII; a single en-US .ftl locale included. This playtest is the first experiment with that arrangement.

3. How the isolation works

- Both binaries resolve their asset roots from the **runtime BEVY_ASSET_ROOT** env var — pointing the server at .../playtest_0009/server and the client at .../playtest_0009/client is the **entire** mechanism. No code changes, no build variants.
- bevy_asset appends assets/ to the root, so the real tree is playtest_NNNN/{server,client}/assets/.... First launch attempt failed on exactly this (paths resolved to .../client/assets/fonts/... with the tree one level up).
- The client’s CommonAssets keys resolve through the **dynamic asset manifest** (collections/common_assets.assets.ron), so a playtest ships its own manifest re-mapping every key to playtest-local files — the keys stay stable, the content is per-playtest.
- Directly-loaded paths the client needs **besides** the manifest keys (found by letting the boot fail and reading the Path not found errors): config.toml (via the asset server), locales/ (a load_folder), fonts/mono/IBMPlexMono-Regular.ttf (the dev console’s), and shaders/{gcd_overlay,crosshair_gcd}.wglsl (HUD materials — plaintext, copied).

4. The plaintext assets

- Server** rigs/playtest_level.gltf: a floor node carrying Skein extras (ColliderConstructor Cuboid, RigidBody: "Static", ClientReplicate) and a spawner node (ClientWorldAsset → the client visuals file, ClientReplicate, PlayerCharacterSpawner) — mirroring the real minimal_level.glb’s Skein structure exactly, zero mesh data.
- Client** visuals/rig/cube/menu/lobby scenes: nodes whose only Skein component is shared::mesh_primitive::MeshPrimitive (a 40×0.5×40 floor, a 2×2×2 cube, a 0.4×1.0 capsule, and camera-carrying menu/lobby backdrops with a sphere/torus for visual distinctness). The glTF cameras block works fine in a text .gltf.
- Locale**: the real en-US .ftl files copied (they are plaintext), plus the new level-playtest-name key appended.

5. What worked

- The full loop runs on the isolated assets**: server up (level metadata + headless glTF instantiation with working colliders), client up through AssetLoading → MainMenu → Lobby → InGame, level list showing only the playtest level, select_level/play driving the real replication flow. Player spawns, grounds, and moves.

- **The menu renders the authored scene:** the sphere backdrop (a MeshPrimitive in a hand-written .gltf, rendered through the scene's own glTF camera) behind the full UI:



Figure 1: GameState::MainMenu on the isolated assets. The gray sphere is the MeshPrimitive-authored backdrop from the plaintext .gltf; the full feathers UI composites over it (the UI_SCALE cut-off even looks milder here than in earlier playtests).

- **The Skein extras deserialization is format-exact:** the first server run accepted the floor node with one wrong field name (half_extents — avian's ColliderConstructor::Cuboid wants x_length/y_length/z_length), logged a clean skein_processing ERROR naming the missing x_length field, skipped the component, and kept running — the player then spawned and **fell through the world forever** (y dropped to -8430 and beyond) while everything else kept working. The failure mode is exactly the documented “authoring mistakes degrade, don't panic” posture, and the ERROR log pinpoints the field. After the field fix: the player grounds on the authored collider.

6. What didn't (all resolved or explained)

6.1. The visuals scene didn't spawn (my own glTF bug)

After adding a second node to the visuals .gltf, the file's scenes[0].nodes read [0, 2] while only nodes 0 and 1 existed — Index out of bounds, the whole asset failed to load, the scene never instantiated (0 mesh entities), and in-game showed only sky. The tell: the CWA entity's WorldAssetRoot **was** present — the get_components probe reports error -23402 (ReflectSerialize missing) for it, meaning present-but-unserializable, see the skill's §6 — while the client log carried the real cause: the glTF validation error. Fixed by correcting the node index.

6.2. The “in-game floor invisible” false alarm (pitch sign)

Even after the fix, look-down screenshots showed only the starfield. That turned out to be the pitch sign convention, not rendering: **look_pitch reads positive when looking up**, and the injected pitch_delta is inverted relative to it — the earlier “look down” injections had been pointing the camera at the sky. At true level pitch the whole scene composites:

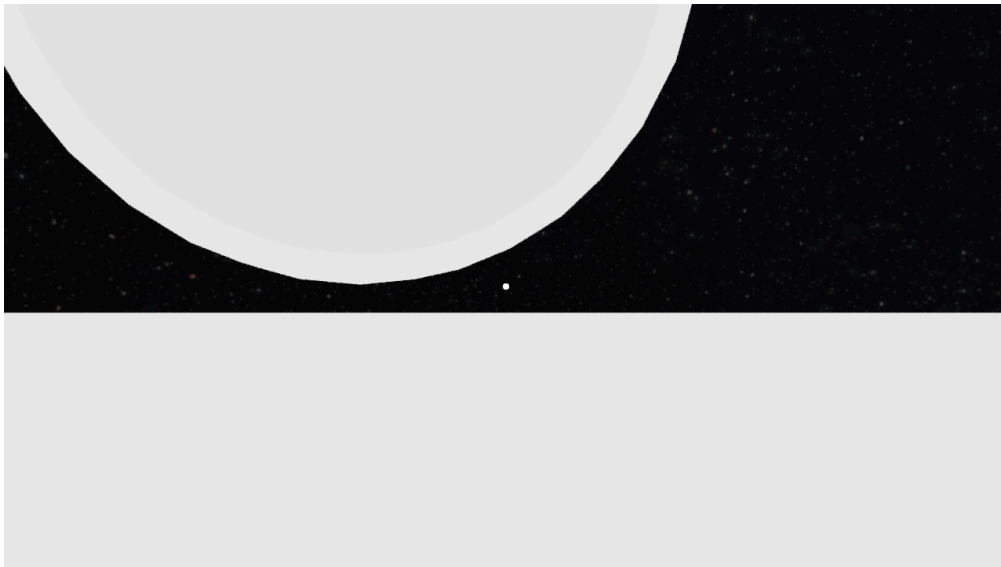


Figure 2: In-game at level pitch: the MeshPrimitive floor (bright under the BRP-injected ambient light), the floating MeshPrimitive sphere ahead, the starfield HDRI skybox, and the HUD crosshair — **all** of it from the isolated plaintext assets (the floor and sphere exist nowhere else).

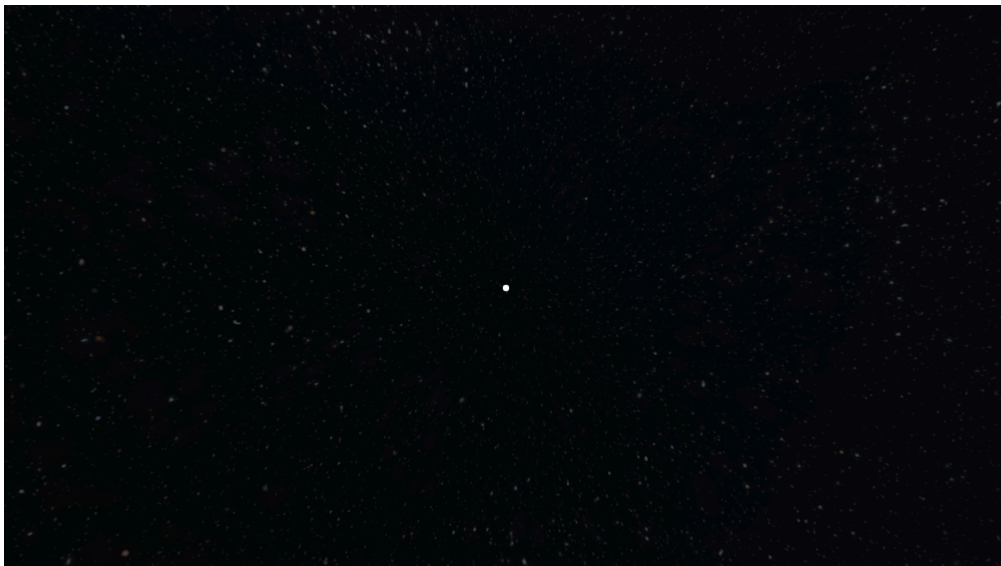


Figure 3: The same scene at `look_pitch +1.5` — the sky-only view that read as “the floor doesn’t render” for most of this run. The pitch convention is documented correctly in `docs/skills/playtest.md` §5 now.

- **Lighting:** the authored `StandardMaterial::default()` is lit-only; with the level’s zero lights the floor is black-on-black (the known minimal-level content gap). Raised the global ambient (`GlobalAmbientLight` via BRP `world.insert_resources, 100 → 5000`) to make the authored geometry visible — a useful playtest technique in its own right, and an argument for giving MeshPrimitive scenes a default light or emissive material option.

7. Findings

- **The isolated-plaintext arrangement works end-to-end** and is the new recommended shape for playtests: `playtest_assets/playtest_NNNN/{server,client}/assets/`, a per-playtest dynamic manifest, Skein-authored logic in text glTF, MeshPrimitive for visuals.
- **Remaining non-text files** are engine furniture CommonAssets requires (fonts, WAVs, the KTX2 skybox, atlas PNGs) — currently copied from `client/assets`. Making them optional would need a feature-gated CommonAssets variant (the feathers embedded default font covers ASCII UI; the HUD’s icon atlas and the audio samples would degrade gracefully) — proposed, not implemented.
- **Bevy’s built-in default font does not remove the font requirement:** CommonAssets loads both font handles through the manifest, and a missing file fails the whole collection (the loading state panics) — the client never gets as far as **using** the default font. The furniture-copy workaround is required at least until that variant exists.

- **Skein extras are authorable by hand in text glTF** with the exact exporter shape (`extras.skein` = an array of `{TypePath: fields} maps`) — verified against the binary `.glb`'s own JSON chunk for ground truth.
- **The camera/UI fixes from playtests 0003/0004 hold on a fully different asset set** — the offscreen pipeline, camera claiming, and UI compositing all worked first-try here.

8. Artifacts & bookkeeping

- Screenshots: curated copies in `docs/playtests/screenshots/playtest_0009/`; the full capture stream in the gitignored `docs/playtests/dist/screenshots/`.
- The assets themselves: `playtest_assets/playtest_0009/` (self-contained; the only non-text files are the copied furniture listed above).
- The tool API surface used: `game/trigger`, `game/levels`, `game/select_level`, `game/state`, `game/input`, `game/screenshot`, plus the BRP builtins for querying and for resource insertion — the ambient-light insert being a new technique this run introduced.